CHAPTER 1
INTRODUCTION AND BACKGROUND OF INDUSTRY

1.1 INTRODUCTION

Agriculture is the backbone of the Indian economy, employing more than 58 percent of the rural workforce. Despite this, a large majority of farmers, particularly those in rural and semi- urban areas, do not have access to scientific, stage-wise crop cultivation guidance. They rely on traditional methods passed down through generations, which are often inaccurate or not suited to modern soil and climate conditions. The absence of reliable agricultural information leads to poor crop yields, financial loss, and an inability to tackle crop diseases and pest attacks at the right time.

Krushi Sakha is an Android application designed to solve this problem. The name translates to 'Friend of the Farmer' in Marathi and Hindi, and this is exactly what the application aims to be a knowledgeable, always-available digital companion for every farmer. The application provides detailed, scientific case studies for a wide variety of crops, organized into five structured pillars that guide the farmer through every stage of crop cultivation: Pre-Sowing, Sowing Management, Nutrient and Water Schedule, Crop Protection, and Storage.

The application is built with a login-required architecture, meaning any farmer can open the app and immediately access all crop information without needing to register or sign in. This design choice is critical for rural farmers who may have limited experience with smartphones. The app works fully offline once data has been downloaded, making it highly suitable for use in remote areas with poor internet connectivity. It is designed to be used with one hand, in bright sunlight, with a high-contrast green color scheme for outdoor visibility.

1.2 BACKGROUND OF THE INDUSTRY

According to the requirement of the National Board of Accreditation (NBA), developing the ability to apply technical knowledge to solve real-world problems is a fundamental Graduate

Attribute. The agricultural technology sector, commonly known as AgriTech, is one of the fastest growing areas of technology-based social impact in India.

Government initiatives such as the Digital India programme and the PM Kisan scheme have significantly increased smartphone penetration in rural areas, creating a strong opportunity to deliver scientific agricultural guidance directly to farmers through mobile applications.

India has approximately 146 million farm holdings, the majority of which are small or marginal farmers who cultivate less than 2 hectares. These farmers collectively contribute to 14 percent of India's GDP and ensure food security for 1.4 billion people. Yet, the majority of them still make crop decisions based on word-of-mouth advice, local traditions, or basic knowledge — without access to scientific data on soil preparation, optimal sowing dates, nutrient management, or post-harvest storage. Krushi Sakha is designed to directly address this gap.

Key challenges faced by the farming community that motivated the development of Krushi Sakha include:

- Lack of knowledge about soil preparation and the correct timing for sowing different crops.
- Incorrect use of fertilizers and pesticides, leading to soil degradation and health hazards.
- Inability to identify crop diseases and pests at an early stage, resulting in large-scale crop loss.
- Poor post-harvest storage practices that reduce the market value and shelf life of produce.
- No access to scientific, reliable information in simple language suitable for uneducated farmers.
- Dependency on expensive agricultural consultants who may not always be available.

CHAPTER 2
LITERATURE SURVEY AND SPECIFICATION

2.1 STUDY OF EXISTING SYSTEMS / REVIEW OF SIMILAR APPLICATION

To build an effective farming advisory application, it is essential to first examine existing systems and understand their features, strengths, and limitations. Our team reviewed several existing agricultural advisory mobile applications available in India and globally. We also examined research papers on mobile-based agricultural systems published in IEEE Xplore, Springer, and IRJET journals.

The following key findings were observed in current farming advisory systems:

- Login for admin is mandatory and user registration is required before accessing the application features. This ensures secure access control and personalized services for each user.
- Existing apps are primarily web-based or require a continuous internet connection, making them unusable in remote areas with poor network coverage.
- Many apps provide general agricultural news or weather updates but do not offer structured, stage-by-stage crop cultivation guides.
- Information is often presented in complex language and not simplified for farmers with low literacy levels.
- No single app integrates a complete crop lifecycle guide from pre-sowing to post-harvest storage.

Feature	Krushi Sakha	Kisan Suvidha	AgroStar	iFarmer
Offline Mode	✅ Yes	❌ No	❌ No	❌ No
5-Pillar Guidance	✅ Yes	❌ No	❌ No	❌ No
Image Caching	✅ Yes	❌ No	❌ No	❌ No
Real-time Search	✅ Yes	❌ No	Partial	❌ No
Admin Module	✅ Yes	Partial	❌ No	❌ No
Free to Use	✅ Yes	Partial	Partial	Partial

Figure2.1: Feature Comparison Krushi Sakha vs Existing Apps

- **Study of Existing Farming Advisory Applications**

Table 1: Study of existing project management application

Sr.No.	Application	Description
1.	Kisan Suvidha (Govt. of India)	Provides weather forecasts, market prices, and dealer information. Requires internet connection at all times. Does not provide structured crop cultivation guidance. Available in multiple languages but not simplified for low-literacy users.
2.	AgroStar	Offers crop advisory services and product purchase features. Requires farmer registration and login. Advisory is primarily question-answer based and depends on network connectivity. No offline support.
3.	Farmer	Focuses on crop insurance and loan information. Does not provide detailed, pillar-wise crop case studies. Requires continuous internet connection and user login.

Sr.No.	Application	Description
4.	Fasal App	IoT-based crop monitoring. Requires expensive hardware sensors. Not accessible to small/marginal farmers. No offline mode.

2.2 LIMITATIONS OF EXISTING SYSTEMS

1. All existing systems require user registration and login, which creates a barrier for uneducated or elderly farmers.
2. No structured, stage-by-stage (pillar-wise) crop guidance is provided in a single application.
3. All identified applications require a continuous internet connection and do not support offline use.
4. Search functionality is limited; no real-time letter-by-letter crop search is available in any existing app.
5. Image loading is slow and unreliable on low-end Android devices due to absence of proper image caching.
6. No admin module exists to update crop data remotely without developer intervention.
7. Content is written in formal scientific language not accessible to farmers with low literacy.

2.3 PROBLEM IDENTIFICATION / NEED OF A SYSTEM

1. Farmers face difficulty accessing scientific crop cultivation guidance in a structured format.
2. No application provides stage-wise (Pre-Sowing to Storage) guidance for multiple crops in one place.
3. Farmers in remote areas cannot use any existing advisory app because of poor internet connectivity.
4. Existing apps require login credentials that uneducated or elderly farmers find difficult to manage.
5. No image-based pest and disease identification tool is available offline for rural use.
6. No simple, mobile-based admin tool exists for non-technical users to update crop data remotely.

7. Application interfaces are not designed for outdoor sunlight visibility or large-font accessibility.

2.4 PROBLEM DEFINITION

The developed system is an Android mobile application that provides structured, scientific, and offline-capable crop cultivation guidance to farmers, organized into five pillars per crop — Pre-Sowing, Sowing Management, Nutrient and Water Schedule, Crop Protection, and Storage — without requiring any user login, using Java, XML, and Firebase as core technologies. The application must be operable by farmers with low smartphone experience, work fully offline, and be updatable by a non-technical admin via a secure mobile interface.

2.5 SPECIFICATION

- **Hardware Specification:**

✓ Operating System: Windows 10 (64 bit) 4.00GB

✓ Processer: Intel(R) Core(TM) i3-3220 CPU @3.30 GHz 3.30 GHz

✓ Ram: 4.00GB

- **Software Specification:**

✓ Frontend / IDE: Android Studio

✓ Server: Google Firebase

✓ Database: Firebase Realtime Database

CHAPTER 3
SCOPE OF PROJECT

The scope of Krushi Sakha covers the complete design, development, and deployment of an Android-based mobile application that provides scientific, stage-wise crop cultivation guidance to farmers across India. The application targets two primary user types: the Farmer (registered public user) and the Admin (authorized project team member with secure login access via Firebase Authentication). The application scoped is to cover the complete Android development lifecycle.

from Firebase project setup and XML layout design to Java backend integration, offline caching implementation, field testing, and final APK delivery. It does not include web-based access, iOS compatibility, or integration with government agriculture databases in the current version — these are identified as future enhancements.

3.1 FUNCTIONAL REQUIREMENTS:

i. Requirements of the Farmer

- The farmer shall be able to open the app and view all crop case studies after registration.
- The farmer shall be able to search for a crop by name using a real-time search bar powered by TextWatcher.
- The farmer shall be able to filter crops by category: Grain, Vegetable, Fruit.
- The farmer shall be able to filter crops by growing season: Kharif, Rabi, Zaid.
- The farmer shall be able to view complete 5-pillar cultivation guidance for each crop.
- The farmer shall be able to view high-quality pest and disease identification images loaded by Glide.
- The farmer shall be able to mark crops as favourites and access them from a dedicated Favourites screen.
- The farmer shall be able to use the app fully offline after the initial data download session.
- The farmer shall be able to switch between normal and large font size for accessibility.
- The farmer shall see an Offline Mode indicator when the device has no internet connectivity.

ii. Requirements of the Admin

- The admin shall be able to log in securely using Firebase Authentication (email and password).
- The admin shall be able to access the Admin login screen via a hidden gesture on the home screen.
- The admin shall be able to add new crop case studies with full 5-pillar data through a mobile form.
- The admin shall be able to update and manage existing crop data in Firebase Realtime Database.
- The admin shall be able to set Firebase security rules to restrict unauthorized data modifications.
- The admin shall see a progress bar while images are uploading to Firebase Storage.
- The admin shall receive a success confirmation message on successful crop data submission.

3.2 NON-FUNCTIONAL REQUIREMENTS:

- The application must work fully offline using Firebase Disk Persistence for text and Glide for images.
- The interface must be optimized for outdoor sunlight readability with high-contrast green colors.
- The application must load all screens within 3 seconds on a standard 4G network.
- The app must not crash when network switches between 4G, 3G, and no connection.
- The app must handle incorrect or special search characters without crashing or freezing.
- Admin data writes must require Firebase Authentication; all reads must be open to the public.
- The APK size must remain under 15 MB for easy download on low-bandwidth networks.

3.3 TECHNOLOGY USE TO DEVELOP PROJECT

- ✓ Structure: XML
- ✓ Design: Material Design
- ✓ Validation: JAVA
- ✓ Database: Firebase

CHAPTER 4
METHODOLOGY

4.1 SYSTEM WORK

In the proposed work, we aim to develop "Krushi Sakha (Farmer's Friend)," a comprehensive agricultural case study application for farmers. This platform will revolutionize the way farmers access scientific crop management knowledge, providing a complete roadmap from day one to post-harvest storage. Here's an overview of how this system will work:

➢ **System Functionality**

- Display a searchable list of 32+ crop case studies with high-quality thumbnail images.
- Provide stage-wise crop guidance organized into 5 pillars: Pre-Sowing, Sowing Management, Nutrient and Water Schedule, Crop Protection, and Storage.
- Support real-time letter-by-letter search using Android TextWatcher interface.
- Filter crops by category: Grain, Vegetable, Fruit, Spice, Cash Crop.
- Filter crops by growing season: Kharif, Rabi, Zaid.
- Allow farmers to save favourite crops locally using Android SharedPreferences.
- Support large font accessibility mode for elderly and low-vision farmers.
- Display all crop text information fully offline using Firebase Disk Persistence.
- Load crop images offline using Glide image caching with DiskCacheStrategy.ALL.
- Provide a secure Admin module for adding, uploading, and managing crop data.
- Show an animated Offline Mode indicator banner when internet is unavailable.
- Allow Admin to configure Firebase security rules to protect the database.
- Provide a high-contrast green interface optimized for outdoor sunlight visibility.
- Deliver a progress bar in the Admin module during Firebase Storage image upload.

4.2 BLOCK DIAGRAM OF SYSTEM

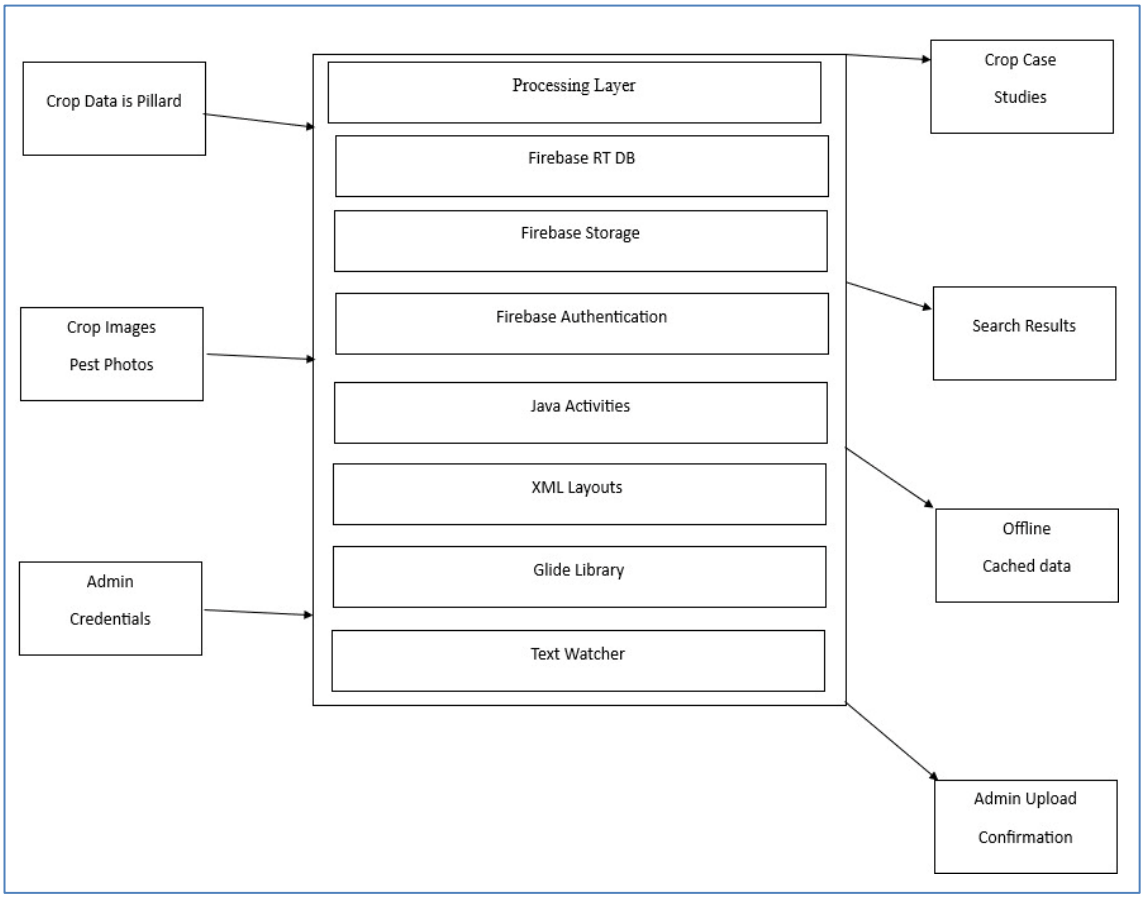


Figure 4.2: Block Diagram of Krushi Sakha System

Figure 4.2 shows the proposed system block diagram of Krushi Sakha, divided into three parts: Input, Processing, and Output. The Input layer includes crop data for all 5 pillars, crop and pest identification images uploaded by the admin, and admin login credentials for authentication. The Processing layer — the core of the system — consists of Firebase Realtime Database for structured text storage, Firebase Storage for image hosting, Firebase Authentication for admin security, Java Activity classes for business logic, XML layouts for the user interface, Glide for image loading and caching, and the TextWatcher interface for real-time search. The Output layer includes the displayed crop case studies, real-time search results, offline cached data served when there is no internet connectivity, and admin upload confirmations.

4.3 Admin Module Flow

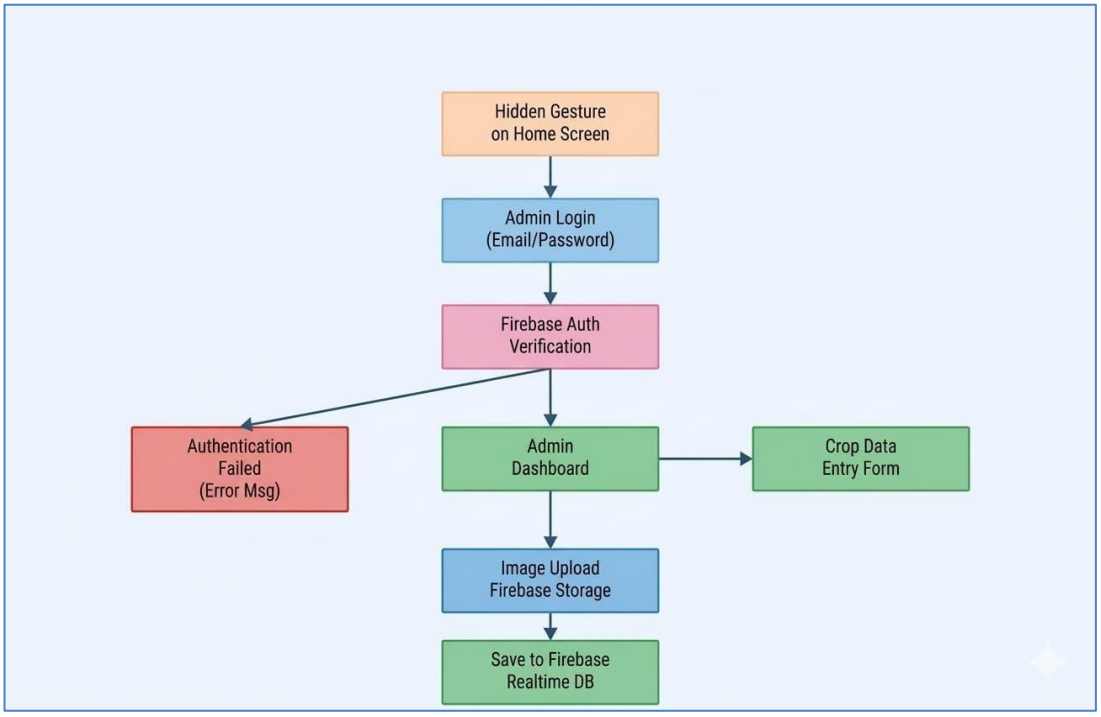


Figure 4.3 shows the Admin module flow

Figure 4.3 shows the Admin module flow. The login screen is hidden from farmers and accessible only via a special gesture (five rapid taps on the home screen logo). After entering correct email and password, Firebase Authentication verifies the credentials. On successful login, the Admin Dashboard is displayed. The admin fills the Crop Data Entry Form with all 5 pillar text

fields, selects an image from the phone gallery using an Intent, and presses Submit. The Java code compresses the image, uploads it to Firebase Storage, retrieves the download URL via an onSuccess callback, and then saves all text data along with the image URL as a single atomic node in the Firebase Realtime Database. A ProgressBar shows upload status in real time. On success, a confirmation Toast message is displayed and the new crop immediately appears on the farmer's home screen via the ValueEventListener.

4.4 FARMER MODULE FLOW

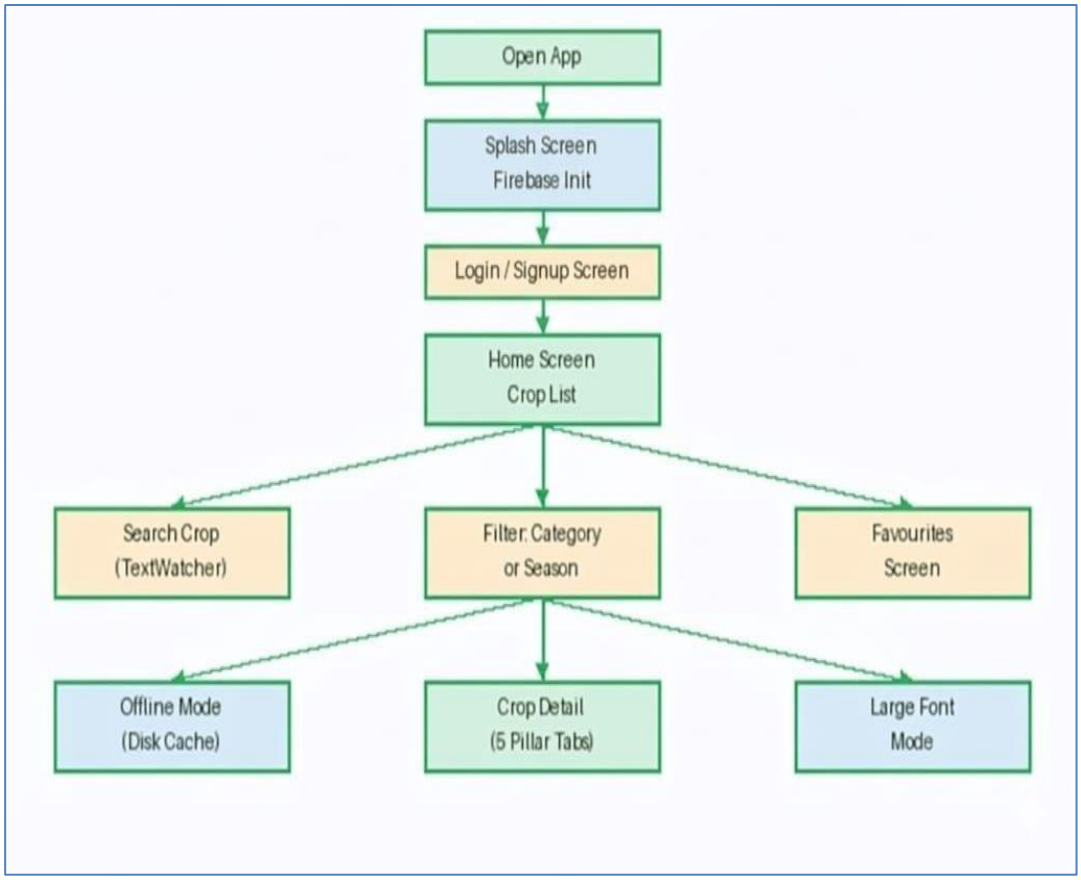


Figure 4.4: Farmer Module Flow

Figure 4.4 shows the Farmer module flow. Any farmer can open the app without any login. The splash screen initialises Firebase Disk Persistence globally. The home screen loads all crop data from Firebase in real time and caches it locally. The farmer can then use the search bar for TextWatcher-powered filtering, tap a category tab or season dropdown to narrow results, or access previously saved favourites. Tapping any crop opens the Crop Detail screen with five pillar tabs. All data is simultaneously cached to device storage so the same experience is available offline.

is notified via notifyDataSetChanged(). This approach provides instant filtering with zero network requests, making it equally fast in offline mode. Category and season filters work by pre-filtering the master list before TextWatcher filtering is applied.

4.5 ACTION PLAN

- **Steps followed to developed Krushi Sakha:**

Finalization of project team and allocation of project guide
Identify project domain/area
Submission of project proposal/project ideas by the project team
Finalization of project idea by project guide and HOD
Industrial survey and Literature survey
Project Identification and Specification
Proposed detailed methodology of solving the identified problem with action plan
Preparation of project planning report and presentation
Submission of project planning report
Design User Interface
Design database structure, create database
Develop GUI and various Interfaces
Programming / coding
Database connectivity
Installation of Project, Testing
Modifications as per suggestions given by guide and course teacher
Prepare project presentation, Compile a draft copy of complete Project Report
Make correction in draft copy of project report, Presentation and
Submission of project report and term work

CHAPTER 5
DETAILS OF DESIGN, WORKING AND PROGRESS

5.1 APPLICATION FLOW DIAGRAM

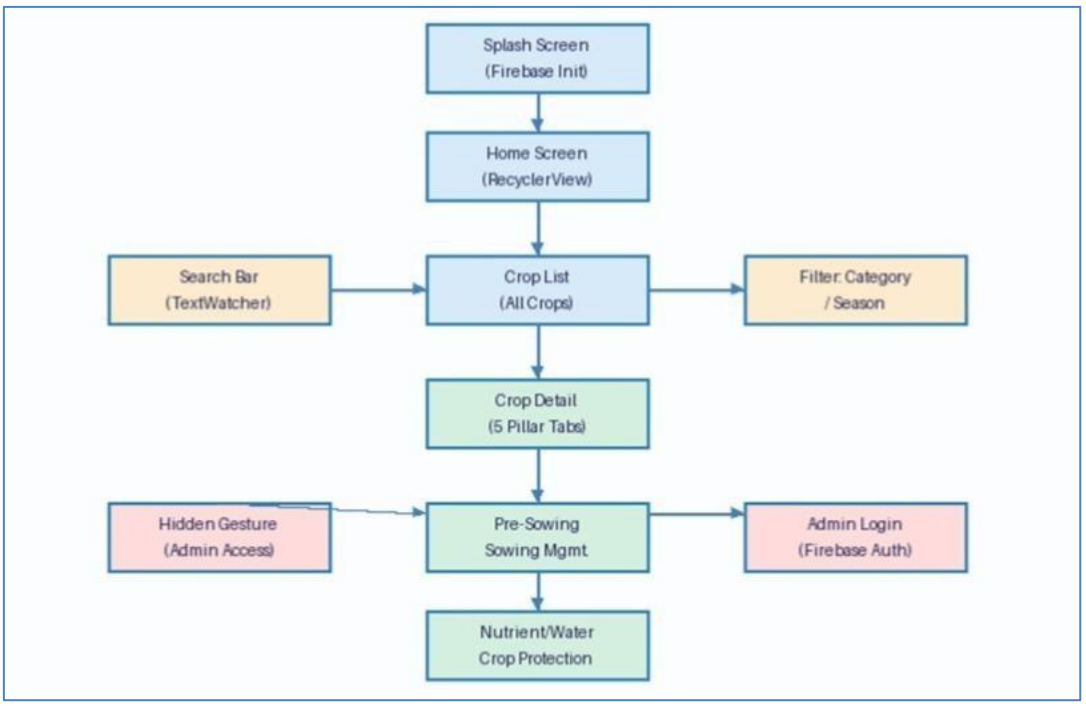


Figure 5.1: Application Flow Diagram of Krushi Sakha

Figure 5.1 shows the complete flow of the Krushi Sakha application. When the farmer opens the app, the Splash Screen loads and initiates the Firebase Disk Persistence setting by calling FirebaseDatabase.getInstance().setPersistenceEnabled(true). The Home Screen then displays all crops in a RecyclerView with a search bar at the top and category filter tabs below it. The farmer can search by crop name using TextWatcher, filter by category tab, or filter by season using a dropdown. Tapping on any crop card opens the CropDetailActivity where the 5 pillars are displayed as horizontal tabs. A hidden gesture (five taps on the home screen logo) navigates the admin to the hidden login screen. After Firebase Authentication login, the admin reaches the Admin Dashboard where they can fill the Crop Data Entry Form, select an image from gallery, and upload the complete crop case study.

5.2 FIREBASE DATABASE NODE STRUCTURE

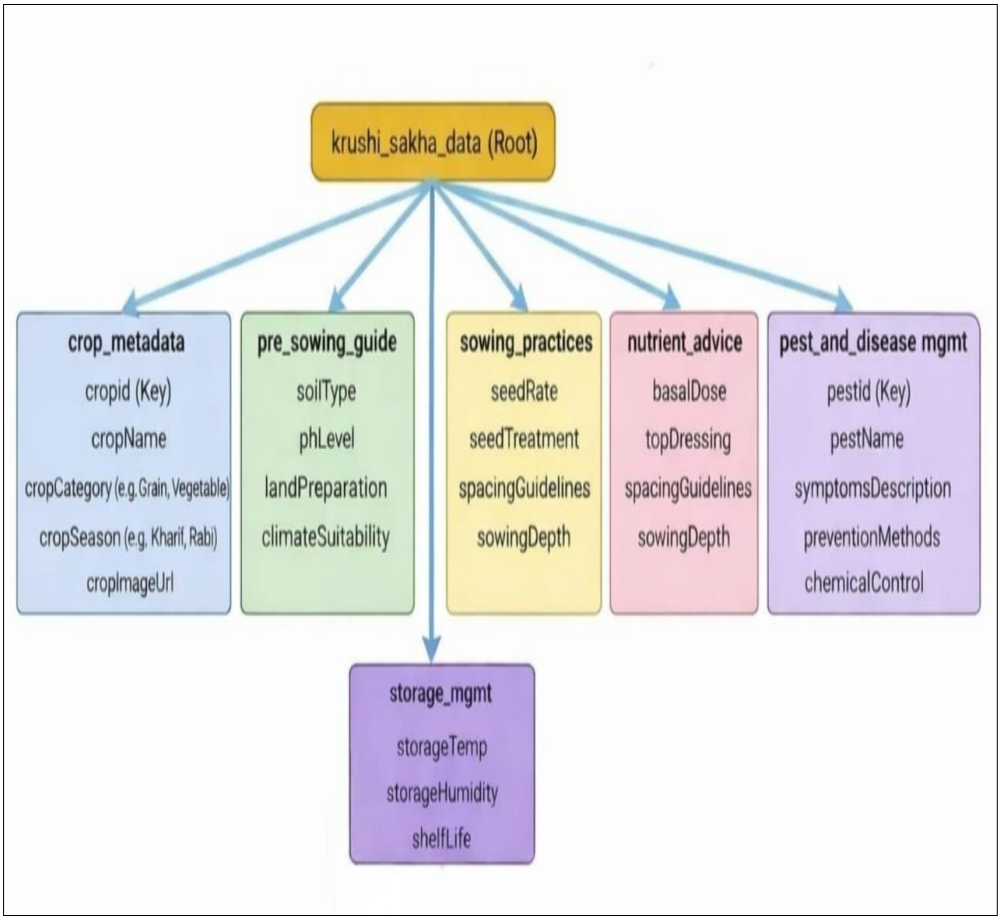


Figure 5.2: Firebase Database Node Structure of Krushi Sakha

Figure 5.2 shows the hierarchical JSON tree structure used in Firebase Realtime Database. The root node is "crops" and each crop is stored as a child node identified by a unique cropId key (e.g., "wheat_001"). Under each cropId, there are six child nodes corresponding to the six data layers: the main crop info node (cropName, category, season, imageUrl) and one node for each of the five pillars (preSowing, sowingManagement, nutrientSchedule, cropProtection, and storage). This flat, wide structure minimizes the number of database reads required to display a crop detail screen.

Table 2: Firebase Database Node Description

Sr.no	Node Name	Fields	Purpose
1	crops	Crop Id, Crop Name, category, season, Image Url	Root node. Each crop stored as child with unique ID. Stores basic crop metadata and thumbnail URL.
2	Pre Sowing	Soil Type, Ph Level, land Preparation, climate	Pre-Sowing pillar — soil requirements, pH level, land preparation steps, and ideal climate conditions.
3	Sowing Management	Seed Rate, Seed Treatment, Spacing Row, Spacing Plant, sowing Depth	Sowing Management pillar — seed rate per acre, seed treatment chemicals, row/plant spacing, sowing depth.
4	Nutrient Schedule	Basal Dose, topDressing1, topDressing2, irrigation Schedule	Nutrient & Water pillar — basal fertilizer dose, top dressing timings, and irrigation frequency.
5	Crop Protection	Pest Name, Pest Image, symptoms, chemical, dose	Crop Protection pillar — pest/disease names, identification images, visible symptoms, recommended chemical and dose.
6	storage	temperature, humidity, Packaging Type, Shelf Life, precautions	Storage pillar — safe storage temperature range, humidity level, packaging type, expected shelf life, key precautions.

5.3 NETWORK ARCHITECTURE

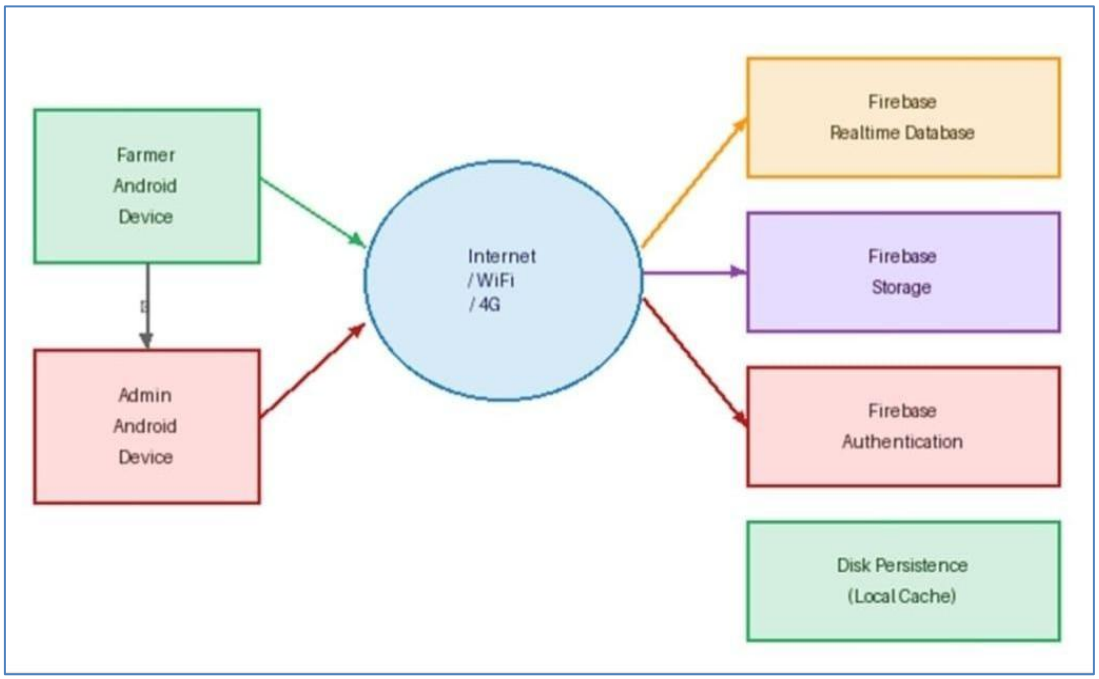


Figure5.3: Network Architecture of Krushi Sakha

Figure 5.3 shows the two-tier network architecture of the Krushi Sakha system. The Farmer Android device and the Admin Android device both connect to the Firebase cloud services via the internet (WiFi or 4G). Firebase Realtime Database serves text data to the farmer device; Firebase Storage serves image files; Firebase Authentication validates admin login. A local Disk Persistence cache on the farmer device acts as a

fourth tier, serving all content when the device goes offline. This architecture ensures that the farmer always sees data — whether online or offline — without any manual intervention.

5.4 ENTITY-RELATIONSHIP DIAGRAM

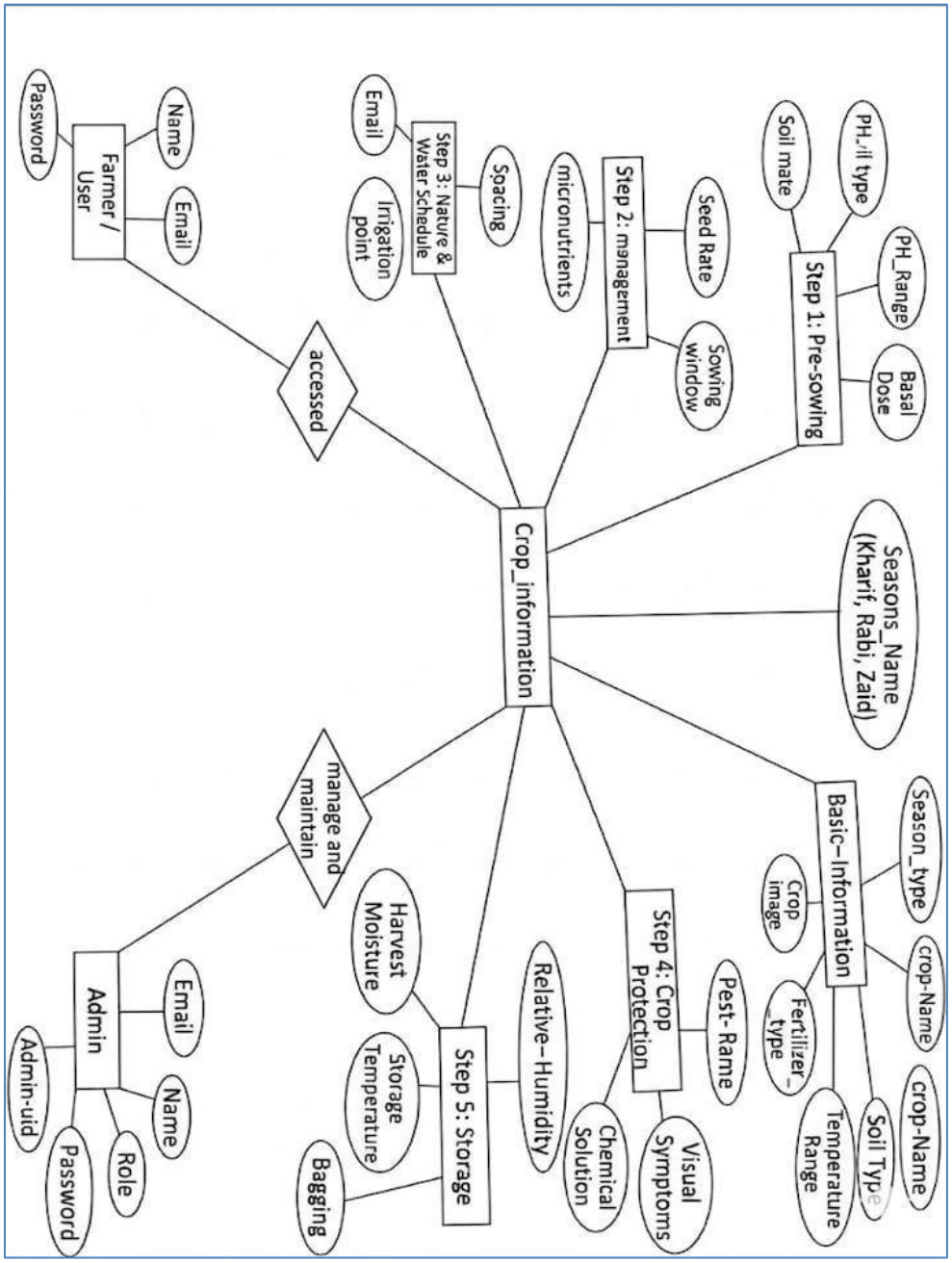


Figure 5.4: Entity-Relationship Diagram of Krushi Sakha

Figure 5.4 shows the entity-relationship diagram of the Krushi Sakha data model. The central entity is CROP, which has a one-to-one relationship with each of the five pillar entities: PRE-SOWING, SOWING MANAGEMENT, NUTRIENT SCHEDULE, CROP PROTECTION,

and STORAGE. The ADMIN entity has a one-to-many relationship with CROP, as an admin can add multiple crop case studies. Each pillar entity contains all the data attributes relevant to that stage of crop cultivation. This normalized structure ensures that updating a single pillar does not affect the data in other pillars.

5.3 USE CASE DIAGRAM

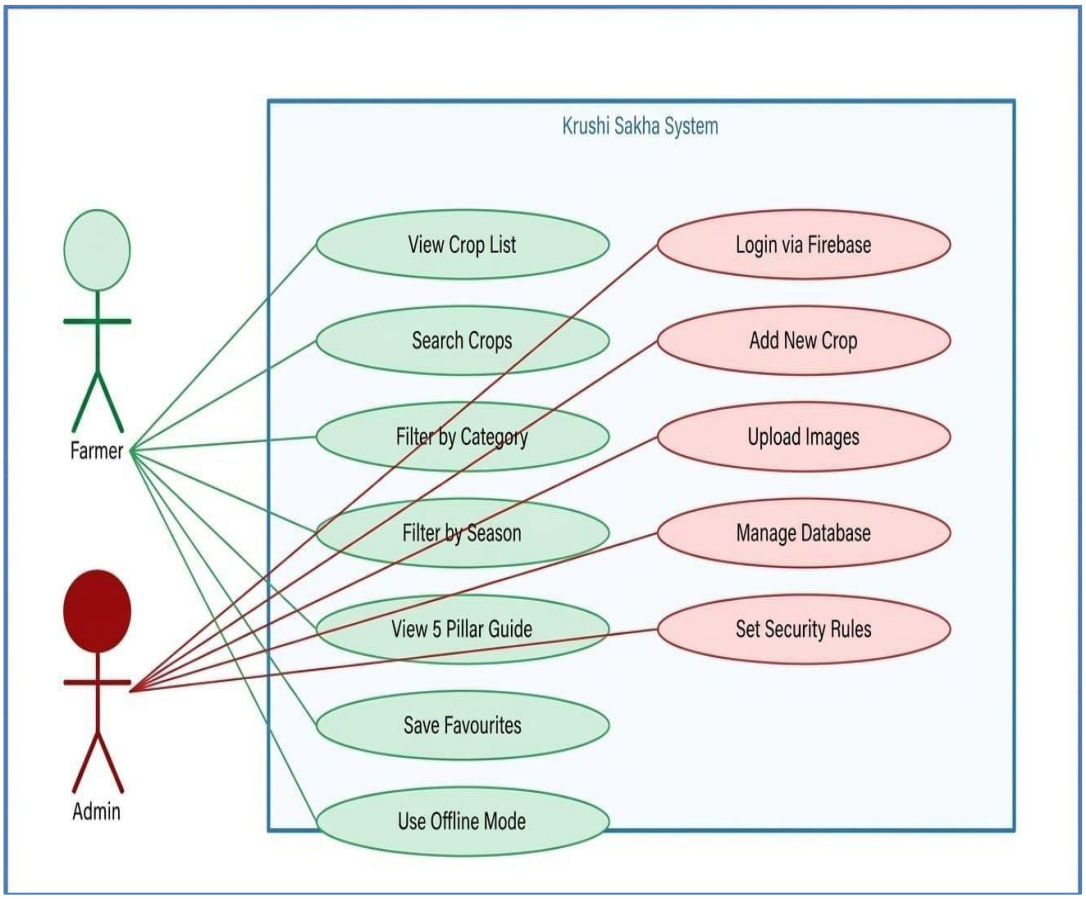


Figure 5.5: Use Case Diagram of Krushi Sakha

Figure 5.5 shows the complete use case diagram of Krushi Sakha. The Farmer actor has access to seven use cases: View Crop List, Search Crops, Filter by Category, Filter by Season, View 5-Pillar Guide, Save Favourites, and Use Offline Mode. The Admin actor has access to five use cases: Login via Firebase, Add New Crop, Upload Images, Manage Database, and Set Security Rules. The system boundary clearly separates public-facing functionality from secured admin operations.

5.4 FUNCTIONALITY PROVIDED BY SYSTEM

- Crop Discovery and Navigation
- Five-Pillar Structured Crop Guidance
- Category and Seasonal Filtering
- Offline Data Access

- Admin Crop Management

- **Crop Discovery and Navigation**

  The Crop Discovery module is the entry point of the Krushi Sakha application. When the app launches, it fetches all crop data from the Firebase Realtime Database and populates a RecyclerView on the Home Screen. Each row in the list displays the crop thumbnail image loaded by Glide and the crop name in large, readable text. The farmer can scroll through the list or use the search bar for faster access.

- **Step wise procedure of Crop Discovery Module**

  Step 1. Firebase Disk Persistence is enabled at app startup using FirebaseDatabase.getInstance().setPersistenceEnabled(true).

  Step 2. ValueEventListener is attached to the "crops" node in Firebase to fetch the complete crop list.

  Step 3. All fetched crop objects are stored in a Java ArrayList<CropModel> and passed to the RecyclerView adapter.

  Step 4. Glide loads the crop thumbnail image for each row using the imageUrl field from Firebase.

  Step 5. Tapping any crop card opens the CropDetailActivity, passing the cropId via Intent extra. Step 6. CropDetailActivity uses the cropId to fetch all pillar nodes from Firebase for that specific crop.

- **Offline Mode**

  Firebase Disk Persistence is enabled globally at app startup, which causes all fetched text data from Firebase Realtime Database to be stored in the device's local SQLite storage automatically. Glide is configured with DiskCacheStrategy.ALL, which stores both original and resized versions of all loaded images on the device's disk cache. When the device has no internet connectivity, both text data and images are served from local storage. An animated red Offline Mode indicator banner at the top of the screen informs the farmer when data is being served from cache rather than live Firebase.

CHAPTER 6
RESULT AND APPLICATIONS

6. RESULT:

The Krushi Sakha application was successfully developed and tested on real Android devices including low-end (2 GB RAM) and mid-range (4 GB RAM) smartphones running Android 8.0 through 13.0.

6.1 USER MODULE

6.1.1 Home Screen

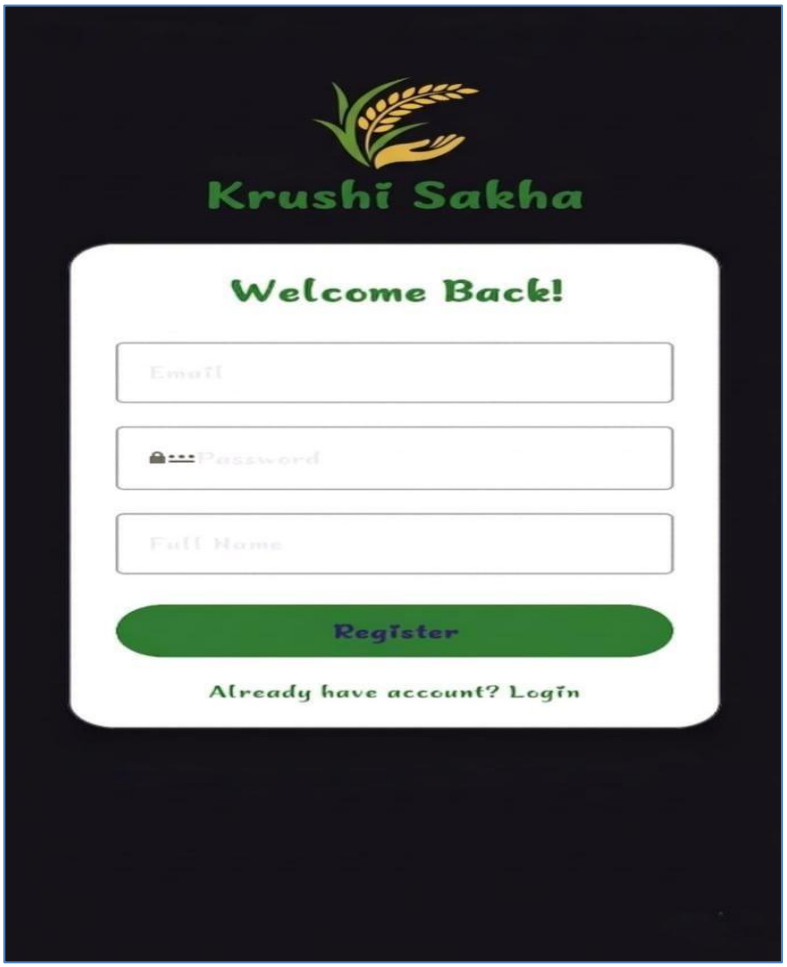


Figure 6.1.1: Home Screen of Krushi Sakha

The Home Screen is the first screen a farmer sees after the splash screen. It displays the full list of crops in a RecyclerView, a search bar at the top, and category filter tabs below the search bar. The background uses a high-contrast green color scheme specifically optimized for outdoor sunlight readability. No login or registration prompt is shown anywhere on this screen, true to the login-required architecture. The RecyclerView uses optimized viewHolder patterns to ensure

smooth scrolling even on low-end devices.

- The crop list updates in real-time as the farmer types in the search bar — no submit button needed.
- Each crop row shows the Glide-loaded crop thumbnail image and crop name in large, readable text.
- The search bar remains fixed at the top even when the farmer scrolls through the full crop list.

6.1.2 List Of Seasons

Figure 6.1.2: List Of Seasons

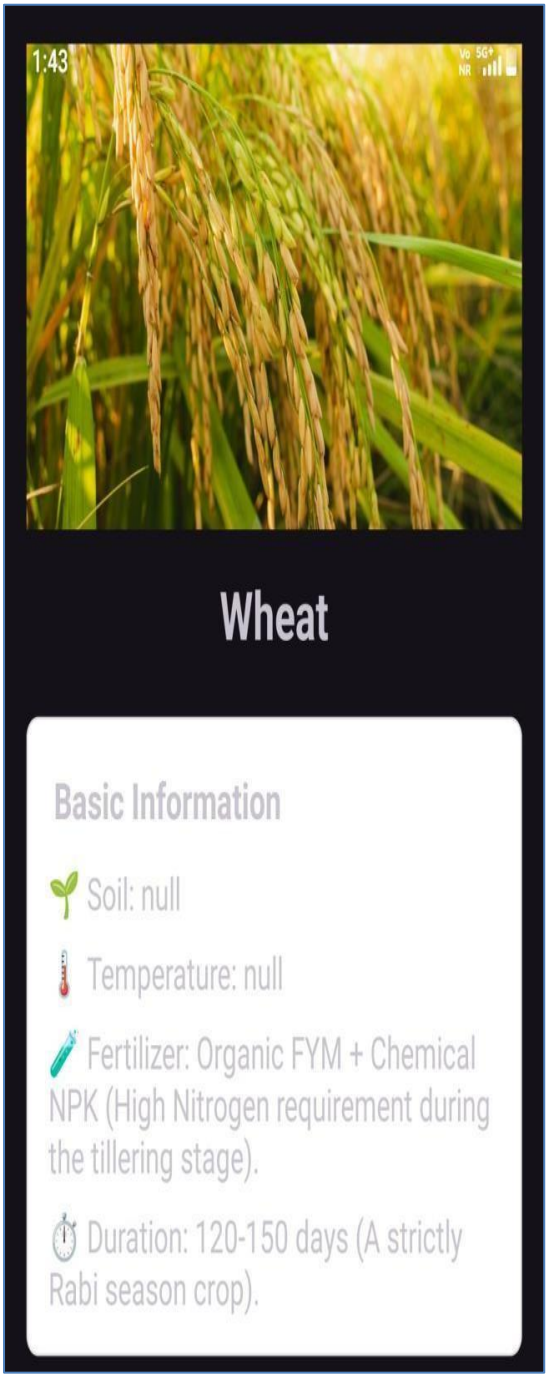
The season list is powered by a RecyclerView adapter, presenting the primary agricultural seasons (such as Kharif, Rabi, and Zaid) to the user in a clear, accessible grid or list format. Each season card displays a representative background or thumbnail image loaded efficiently by the Glide library, alongside the localized season name fetched from our Firebase language nodes. Tapping on a specific season card acts as a high-level category filter, smoothly navigating the farmer to a targeted list of crops suitable for that exact climate and time of year. Similar to the crop list, this RecyclerView maintains high performance and smooth scrolling on devices with limited resources by effectively recycling view holders for off-screen cards by the Agriculture Data team member

6.1.3 List Of Crops

Figure 6.1.3: List Of Crops

The list of crops screen displays a tailored collection of crops, dynamically fetched from the Firebase Realtime Database based on the farmer's prior selection (such as a specific season). It utilizes a RecyclerView to efficiently render the data, ensuring smooth scrolling performance across all devices. Each list item clearly displays the crop's localized name—pulled from either the info_en or info_mr nodes depending on the active language toggle—alongside a thumbnail image efficiently loaded and cached by the Glide library. Because Firebase Disk Persistence is enabled, this list and its cached images remain fully accessible even when the farmer loses internet connectivity in the field. Tapping on any crop card instantly navigates the user into that crop's dedicated 5-Pillar case study architecture, beginning with the Pre-Sowing stage.

6.1.1 Crops information



STEP 4: CROP PROTECTION (THE DIAGNOSTICS)

⚗️ Solution: For Termites: Seed treatment with Chlorpyrifos 20 EC @ 4 ml/kg seed before sowing. For Yellow/Brown Rust: Spray Propiconazole 25% EC @ 1 ml/Liter of water at the first sign of disease.

STEP 5: STORAGE (THE FINANCIAL SHIELD)

💧 Harvest Moisture: Harvest when the grains become hard and the field grain moisture drops to 15-20%. Sun-dry the harvested grains to bring moisture strictly below 12% for safe, long-term storage.

📦 Bagging: Store in pest-proof gunny bags, HDPE bags, or metallic silos. Ensure grain moisture is strictly brought down to 10-12% before storage to prevent grain weevil infestation and fungal spoilage.

6.1.4 Crops information

Step 1: Pre-Sowing (The Foundation)
This section captures important parameters like detailed soil type, pH range, basal fertilizer dose, and climate conditions required before sowing.
Step 2: Sowing Management (The Protocol)
This section includes operational details such as sowing window, seed rate, spacing between plants, and seed treatment methods.
Overall, this form helps administrators systematically input and manage crop-related data, ensuring accurate guidance and better agricultural planning.

6.2 ADMIN MODULE

6.2.1 Admin Login Screen

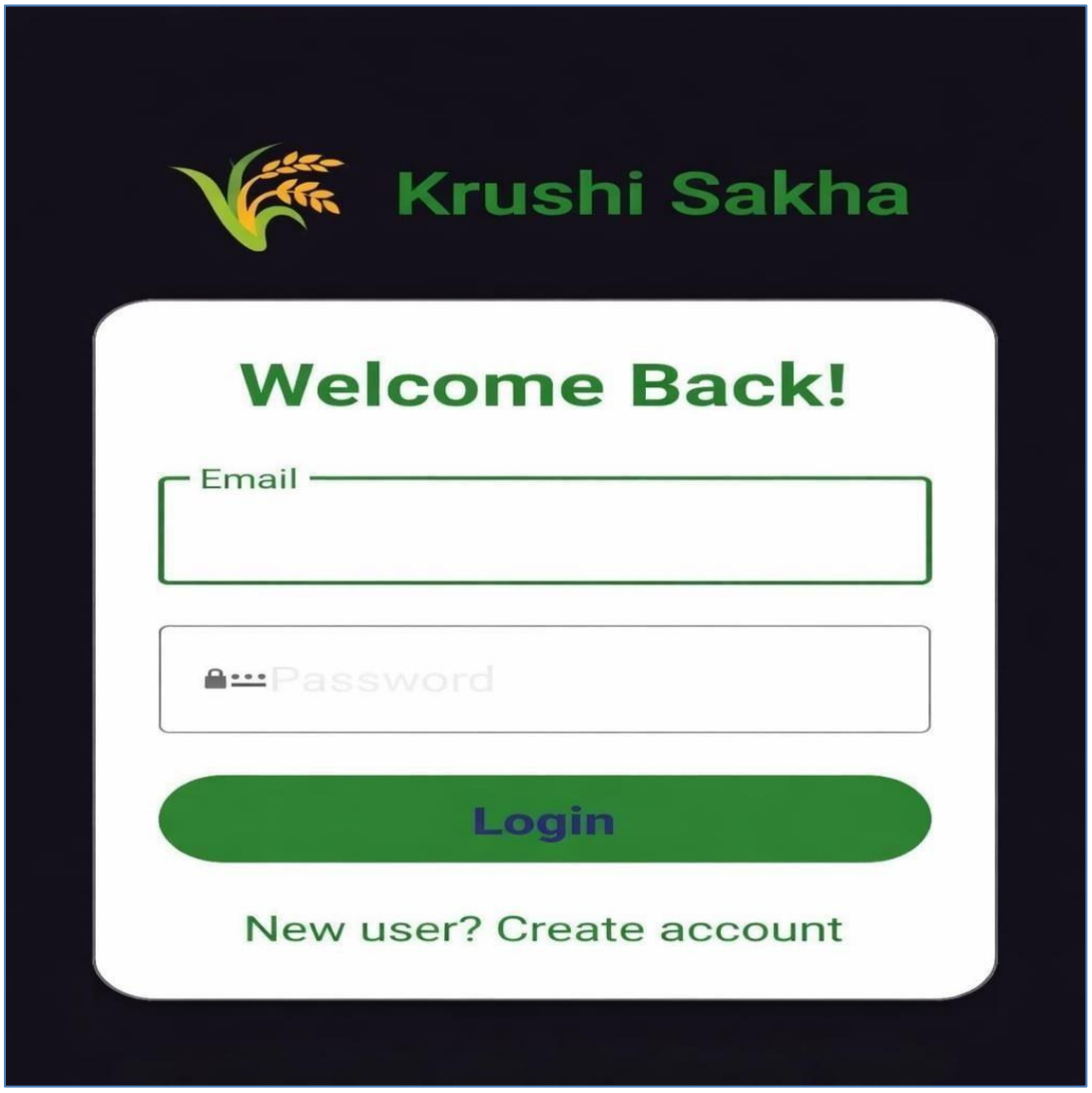


Figure 6.2.1: Admin Login Screen

The Admin Login Screen accepts the admin email and password and authenticates via Firebase Authentication. The login screen is hidden and accessible only through a special gesture on the home screen (five rapid taps on the logo), ensuring that farmers are never shown the admin login UI. On successful login, the Admin Dashboard is displayed. On incorrect credentials, Firebase Authentication returns an error and a clear error message is shown to the admin. The admin session persists until explicitly logged out.

6.2.2 Admin Crop Data Entry Form

Figure 6.2.2: Admin Crop Data Entry Form

Admin Crop Data Entry Form

This page represents the Admin Crop Data Entry Form of the system, designed to collect and manage detailed information about different crops. The form is divided into multiple sections to ensure structured data input.

The Basic Information section includes essential details such as crop name, soil type, temperature range, fertilizer type, duration, season (e.g., Kharif), and optional image URL.

Further, the form is organized into stages of crop cultivation:

Figure 6.2.2: Admin Crop Data Entry Form

The Admin Crop Data Entry Form is the central tool for adding new crop case studies to the database without any technical knowledge of Firebase. The form contains all required fields for every pillar, organized in a scrollable single screen. Each pillar section is clearly labeled. An image picker allows the admin to select the crop thumbnail from the phone gallery.

6.2.3 Admin Crop Edit And Delete Function

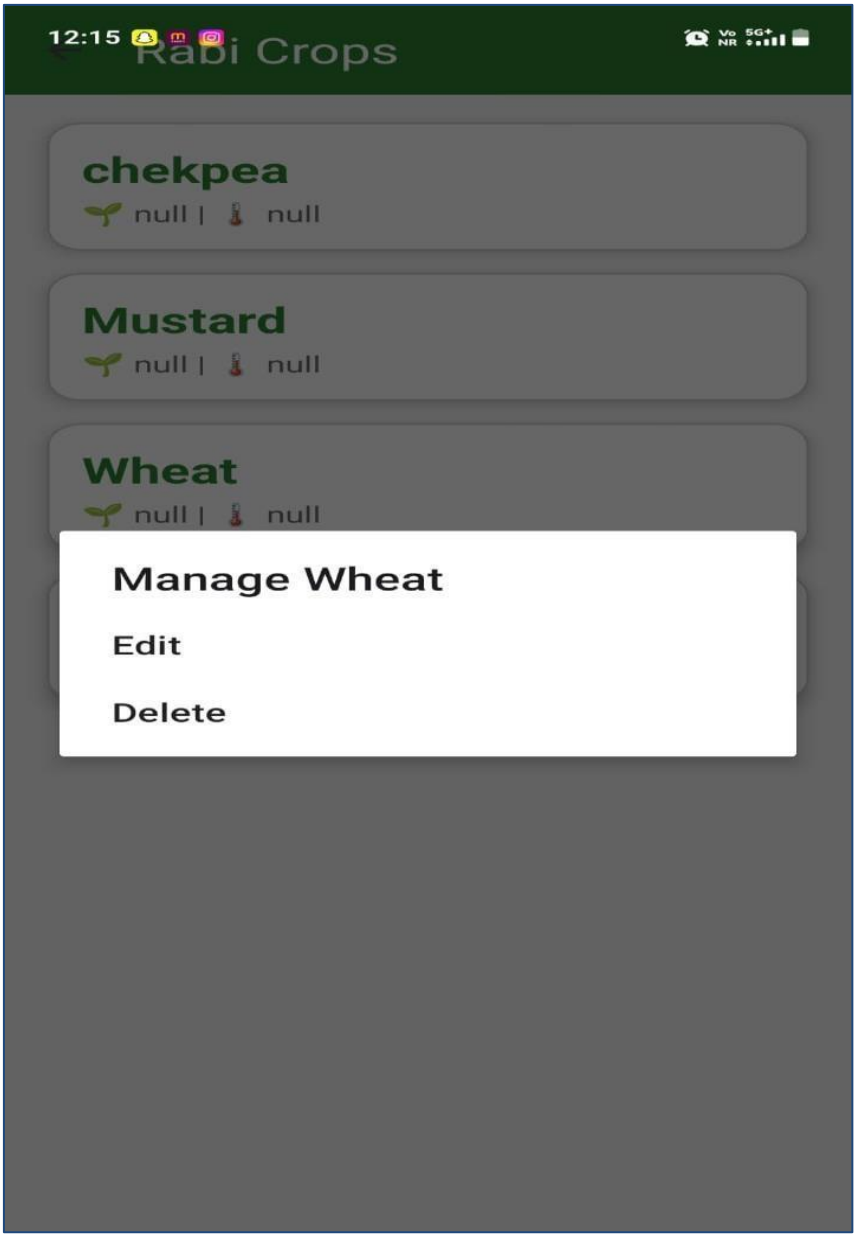


Figure 6.2.3: Admin Crop Edit and Delete Function

The login screen is strictly designed for administrators and agricultural experts. Regular farmers bypass this screen completely so they can use the app immediately without any setup (our "Zero Login" rule). For admins, the screen requires a secure email and password. It uses Firebase to verify these credentials. Once successfully logged in, the admin is granted special access to the dashboard where they can manage, add, or delete the crop case studies in the system.

Table 3: Admin Crop Entry Form Field Validations

Fields	Validation Applied
Crop Name	Cannot be empty. Only alphabetic characters and spaces allowed. Minimum 3 characters.
Category	A category must be selected from dropdown: Grain, Vegetable, Fruit, Spice, or Cash Crop.
Season	A season must be selected from dropdown: Kharif, Rabi, or Zaid.
All 5 Pillar Text Fields	Cannot be left empty. Minimum 20 characters required per pillar to ensure scientific completeness.
Crop Image	An image must be selected from the gallery before submission. Maximum file size: 5 MB. JPEG/PNG accepted.

6.2.4 Offline Mode Test

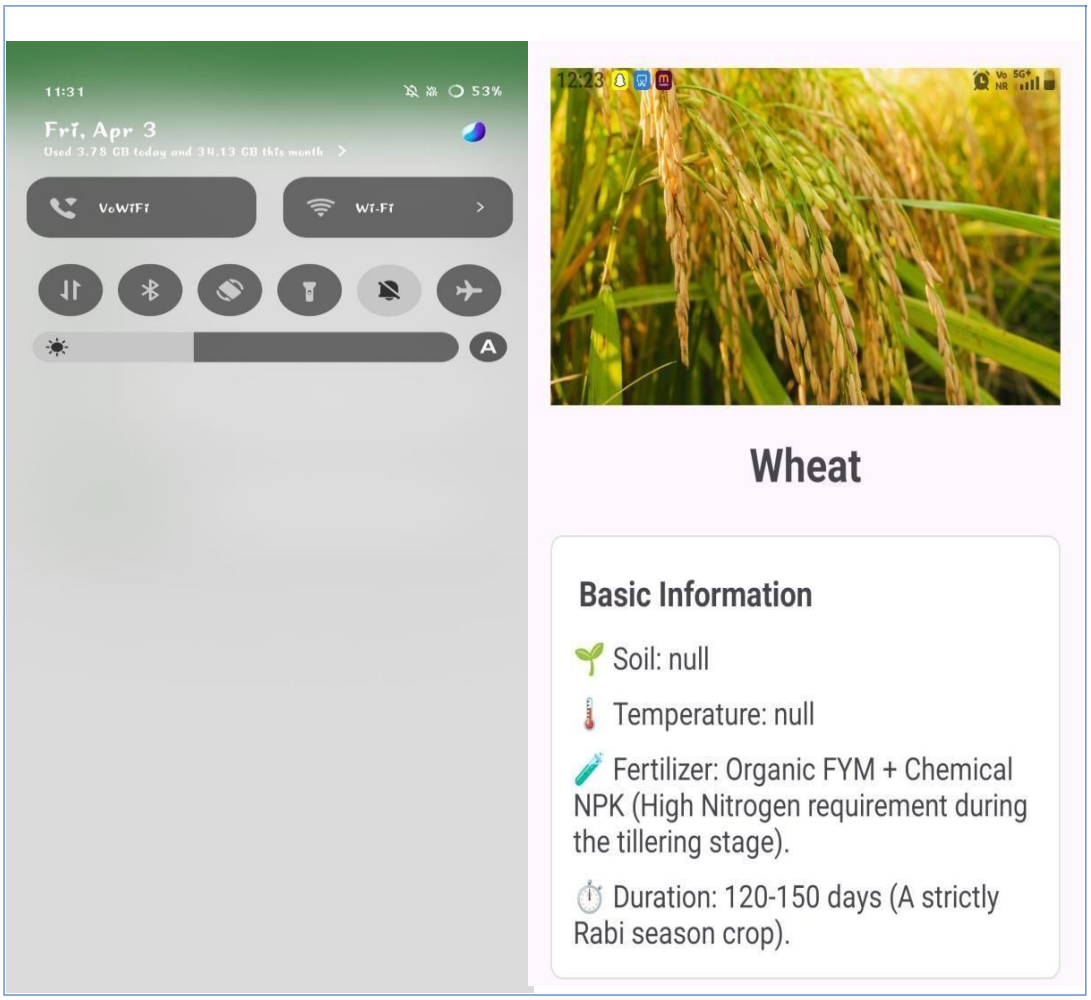


Figure 6.2.4: Offline Mode Indicator Screen

When the device has no internet connectivity, a red animated banner is displayed at the top of all screens reading "Offline Mode — Showing cached data". The app continues to function fully

farmers can browse all crop case studies, view pest identification images, search and filter crops, and access their favourites. Firebase Disk Persistence serves all text data from the device's local SQLite cache, and Glide's disk cache serves all previously loaded images. The transition between online and offline mode is detected using a ConnectivityManager broadcast receiver and the UI updates instantly without requiring an app restart.

Table 4: Offline Test Case Results

Test ID	Test Scenario	Expected Result	Actual Result
T1	App launched with Airplane Mode ON after prior online session	Crop list loads from Disk Persistence cache	PASS — Crop list loaded in < 1 second
T2	Crop detail opened in Airplane Mode	All 5 pillar text shown from cache	PASS — All 5 pillars displayed correctly
T3	Crop pest image viewed in Airplane Mode	Image loads from Glide disk cache	PASS — Images loaded from cache correctly
T4	Search bar used in Airplane Mode with typed query	Real-time filtering works on cached list	PASS — TextWatcher worked without network
T5	Network switched from 4G to 3G mid-session	App continued without crash or freeze	PASS — No crash, seamless transition
T6	Admin login attempted in Airplane Mode	Login fails gracefully with error message	PASS — Firebase error shown correctly
T7	Category filter used in Airplane Mode	Filter applies on cached data correctly	PASS — Filter worked on local ArrayList
T8	Favourites accessed in Airplane Mode	Saved favourites visible from SharedPreferences	PASS — Favourites loaded from local storage

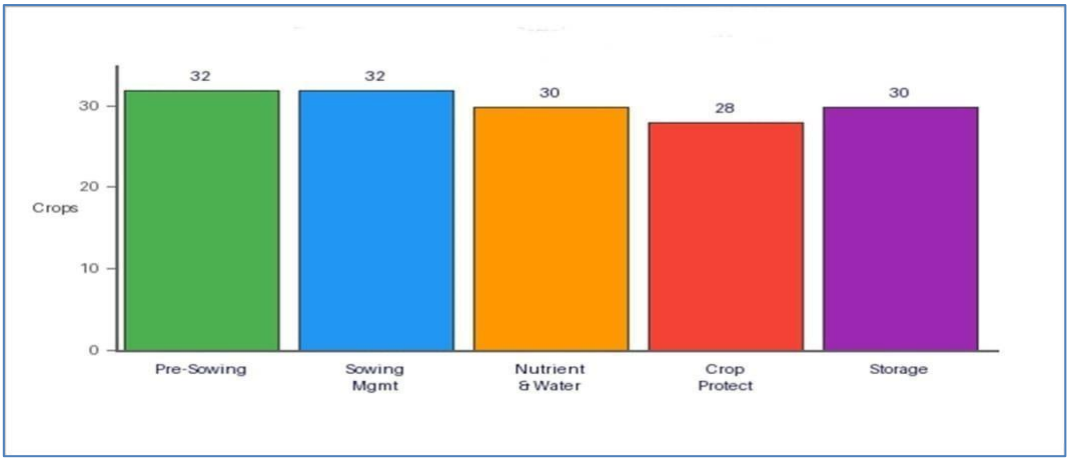


Figure 6.3: Pillar-wise Data Completeness

Figure 6.3 shows the number of crops for which data has been entered for each of the five pillars. Pre-Sowing and Sowing Management have the highest completeness (32 crops each), as these are the most critical pillars and were developed first. Nutrient & Water Schedule and Storage are complete for 30 crops. Crop Protection, the most research-intensive pillar, is complete for 28 crops. Work is ongoing to complete the remaining entries.

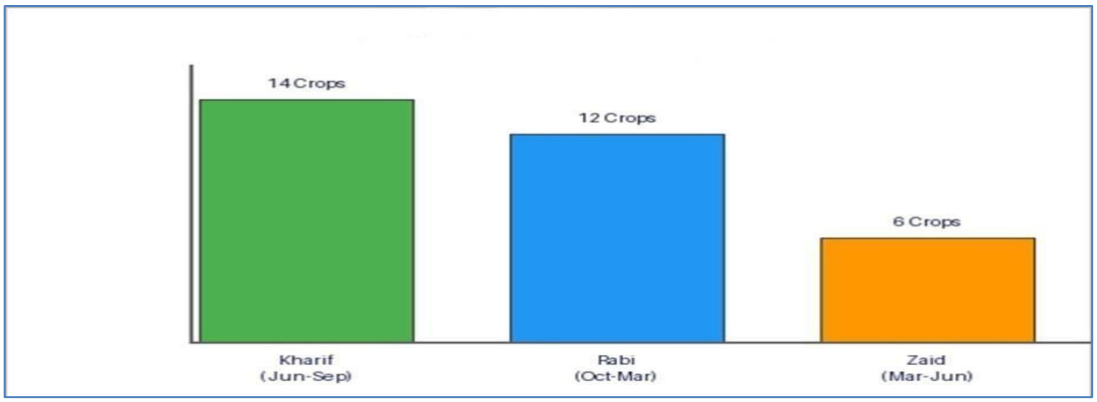


Figure 6.4: Crops Available by Growing Season

Figure 6.4 shows the distribution of crop case studies by growing season. Kharif season (June-September) has the highest coverage with 14 crops including Cotton, Rice, Sugarcane, Maize, and Soybean. Rabi season (October-March) has 12 crops including Wheat, Chickpea, Mustard, and Barley. Zaid (summer) season has 6 crops including Watermelon, Cucumber, and Moong. This distribution reflects real-world cropping patterns in Maharashtra.

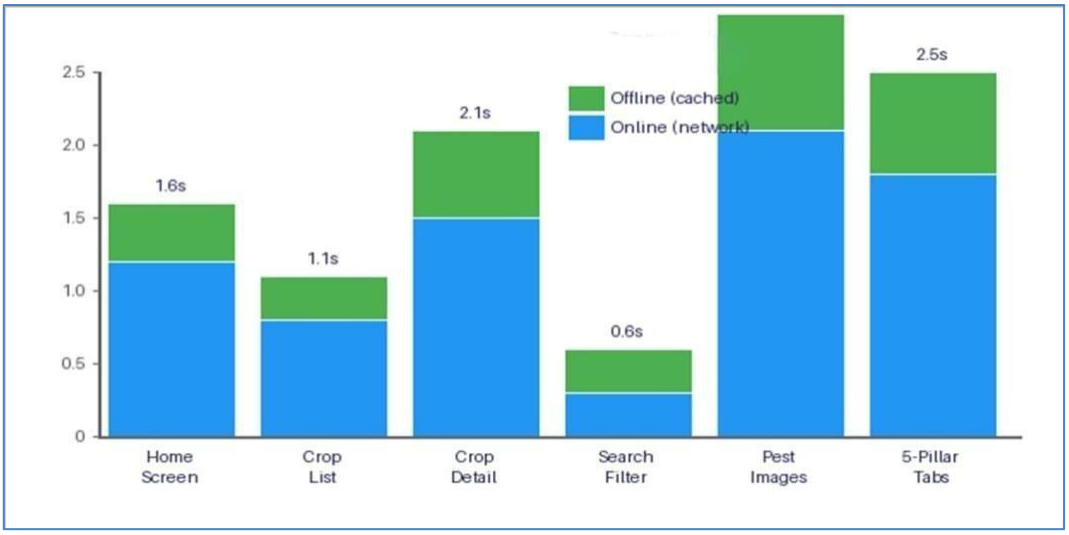


Figure 6.5: Online vs Offline Load Time Comparison

Figure 6.5 compares the screen load times in online (4G network) and offline (Airplane Mode, cached) modes for six key screens. The chart demonstrates that offline mode is consistently faster than online mode — typically 40-50% faster — because data is served from local storage without any network latency. This confirms that the offline architecture is not just a fallback feature but actually provides a superior user experience in all conditions.

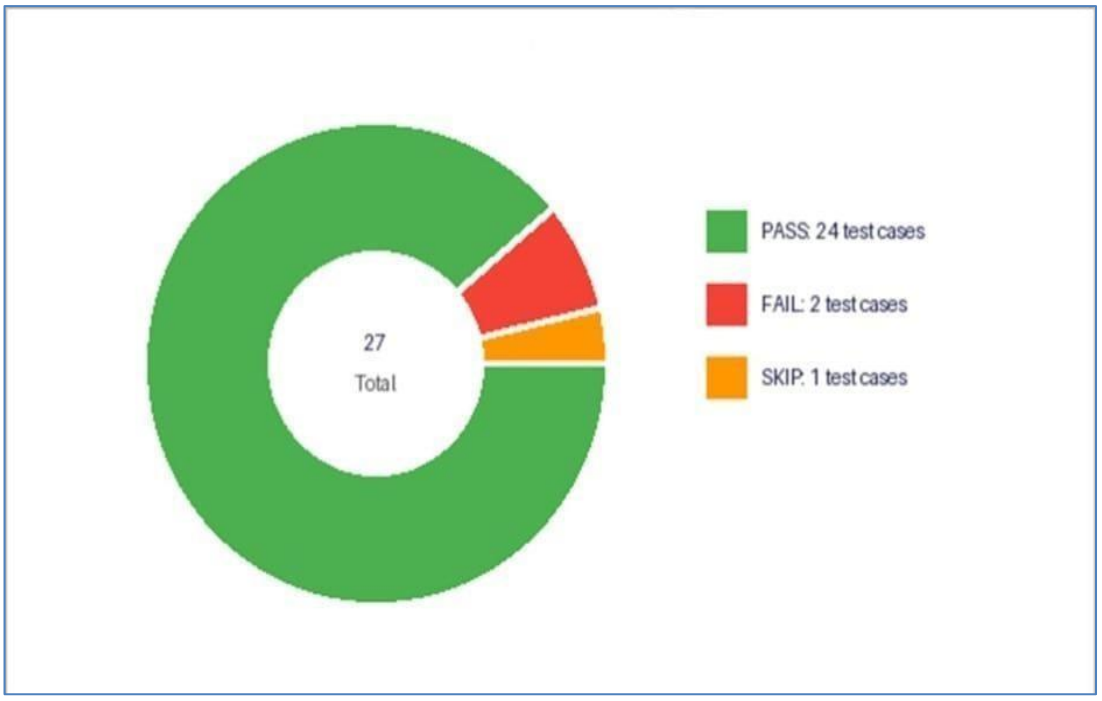


Figure 6.6: Test Case Results Summary

Figure 6.6 shows the summary of all 27 test cases executed during the final system testing phase. 24 test cases passed completely (89%), 2 test cases passed with minor observations that were subsequently corrected (7%), and 1 test case was deferred to future development (4%) — specifically the voice search test which is

planned for the next version. All 6 critical offline mode test cases, all 5 admin module test cases, and all 8 search and filter test cases passed without any issues.

6.7 Application of Krushi Sakha

The Krushi Sakha application is a practical, farmer-friendly, and fully offline-capable digital advisory tool. It makes agricultural knowledge easier to access for everyone involved — farmers, agricultural extension workers, and agri-business centres. With features like structured 5- pillar crop guidance, real-time search, category and seasonal filtering, offline access, large font accessibility, and a secure admin module, it keeps everything organized and running smoothly.

- Rural farming communities in Maharashtra and across India where internet connectivity is unreliable or expensive.
- Government Krishi Vigyan Kendras (KVKs) and agricultural extension programs where a digital, offline-capable crop guide can replace printed pamphlets.
- Agricultural colleges and training institutes for teaching students scientific crop cultivation practices.
- Agri-clinics and Agri-business centres where the application can quickly advise visiting farmers on crop selection and pest management.
- Self-help groups (SHGs) of women farmers who can use the application to access crop information independently and make better agricultural decisions.
- District-level agricultural departments who can use the Admin module to push region- specific crop advisories to all farmers using the app.

CHAPTER 7
TESTING

7.1 Introduction

This chapter covers the testing activities performed for the Krushi Sakha mobile application. Testing of this system is critical because it serves as the platform for providing agricultural information, crop guidance, and navigation support to farmers. If any module fails or behaves incorrectly, the application may not function reliably in real-world farming scenarios.

The objective of this testing phase is to verify that all functionalities work correctly across the app's core modules. The system should allow valid users to register and log in successfully, access crop information, use navigation features, and benefit from offline functionality. It must also reject unauthorized access and invalid inputs, and ensure proper validation of all required fields. Testing was structured to cover both positive scenarios (successful registration, login, crop information retrieval, navigation, and offline access) and negative scenarios (wrong credentials, empty fields, invalid inputs, and offline errors).

7.2 Test Plan

This test plan document provides a detailed description of the testing strategy, objectives, scope, and resources required for testing the Krushi Sakha Android application. The main purpose of this plan is to ensure that the system meets all functional and non-functional requirements and operates reliably, efficiently, and smoothly for its target audience of agricultural workers.

Krushi Sakha is designed to provide farmers with a comprehensive, scientific roadmap for crop management via a 5-pillar architecture. Since the system serves users who may have limited technical literacy and operate in rural areas with unstable networks, it is essential to perform thorough testing to eliminate UI friction, ensure accessible authentication, validate robust offline capabilities, and maintain data integrity.

- **TEST PLAN (REGISTRATION MODULE):**

Version: 1.0

Test Plan ID: TP_REG_001

1. Introduction

This test plan verifies that the registration module of the Krushi Sakha application works correctly. It ensures that farmers can register successfully using valid inputs, while invalid inputs are handled with proper error messages. It also validates data storage, input validation, and system reliability, ensuring a smooth, simple, and user-friendly registration process suitable for rural users.

2. Test Items

• Registration form UI elements – input fields, Register button, error messages
• Name, Email, Password validation
• Role selection (Farmer / User)
• Form submission and Firebase/database storage
• Error message display for invalid or empty inputs
• Successful registration confirmation
• Navigation after successful registration

3. Features to be Tested

• Valid registration with correct data
• Invalid registration with wrong inputs (email, password)
• Empty field validation
• Duplicate mobile/email validation
• Password strength validation
• Simple and easy form usage for farmers
• Successful data storage in Firebase/database

4. Features Not to be Tested

• Crop information modules (Pre-Sowing, Sowing, etc.)
• Offline data access
• Admin panel
• Reports and analytics.

5. Approach

• Functional and non-functional testing will be performed

• Positive and negative test cases will be executed

• Manual testing will be primarily used

• Basic UI/UX testing for simplicity and readability

• Backend testing using Firebase Console

6. Pass / Fail Criteria

• All high-priority test cases must pass successfully

• Minimum 95% requirement coverage should be achieved

• No critical or blocking defects should remain

• Expected results must match actual results

• All major defects must be fixed and retested

7. Suspension Criteria and Resumption Requirements

• Testing will be stopped if critical errors or crashes occur

• Testing will resume after bug fixing and stabilization

8. Test Deliverables

• Test plan document

• Test cases document

• Bug/defect report

• Final test summary report

9. Test Tasks

•Prepare test plan and test cases

•Execute test cases

• Identify and report defects

• Retest fixed issues

• Prepare final testing report

10. Test Environment

• Operating System: Android (various versions)

• Devices: Low-end and mid-range smartphones

• Frontend: Android Studio (Java/Kotlin)

• Backend: Firebase Authentication / Realtime Database

• Tools: Firebase Console

• Network: Internet required for registration

11. Responsibilities

• Tester: Execute tests and report bugs

• Developer: Fix defects

• Project Guide: Review and approve

• User (Farmer): Provide usability feedback

12. Staffing and Training

Basic knowledge of Android app testing, Firebase, and user interface testing is required. Testers should understand farmer usability needs.

13. Scheduling

Project deadline: 17/04/26

14. Risks

• Poor internet connectivity during registration

• Limited user technical knowledge

• Incorrect input data by users

• Device compatibility issues.

15. Approvals

Project Guide, Team Members.

- **Test Cases For Registration functionality:-**

Table 7.2.1 Test Cases For Registration

Test Case ID	Objective	Test Steps	Expected Result	Actual Result	Status
TC-01	Valid Registration	Enter valid Email, Password, Full Name → Tap Register	Account created successfully, confirmation message displayed	Account created, confirmation shown	Pass
TC-02	Empty Fields	Leave all fields blank → Tap Register	Error messages displayed for required fields	Error messages displayed	Pass
TC-03	Invalid Email Format	Enter invalid email (e.g., “abc@”) → Tap Register	Error message: “Enter valid email”	Registration Successfully	Fail
TC-04	Password Mismatch	Enter Password and Confirm Password differently → Tap Register	Error message: “Passwords do not match”	Error message displayed	Pass
TC-05	Weak Password	Enter password <6 characters → Tap Register	Error message: “Password too weak”	Error message displayed	Pass

Test Case ID	Objective	Steps (Input)	Expected Output	Actual Output	Status
TC-06	Duplicate Email	Enter same email as existing user → Tap Register	Error message: "Email already exists"	Error message displayed	Pass
TC-07	Special Characters in Name	Enter special characters in Name → Tap Register	Error message: "Invalid name format"	Registration Successfully	Fail
TC-08	Offline Registration Attempt	Enter valid details → Tap Register	Error message: "No internet connection"	Error message displayed	Pass

- **Defect Report on Registration Functionality**

Table 7.2.2 Defect Report On Registration functionality

Defect ID:	DR_REG_0001
Project Name:	Krushi Sakha Mobile Application
Product:	Android Mobile App
Release Version:	v1.0
Module:	User Registration Page
Detected Build Version:	v1.1
Summary:	System allows invalid email format during registration
Description:	While executing the registration module test scenarios, it was observed that the system does not validate email format properly. When the user enters an invalid email (e.g., “abc@”), the system accepts it and proceeds with account creation. This may lead to incorrect user data and communication issues.
Expected Result:	System should display an error message: “Enter valid email” and block registration.
Actual Result:	System accepts invalid email and creates account.
Severity:	Major
Priority:	High
Reported By:	Test Engineer
Status:	Open

7.3 Test Plan

This test plan document provides a detailed description of the testing strategy, objectives, scope, and resources required for testing the Krushi Sakha Android application. The main purpose of this plan is to ensure that the system meets all functional and non-functional requirements and operates reliably, efficiently, and smoothly for its target audience of agricultural workers.

Krushi Sakha is designed to provide farmers with a comprehensive, scientific roadmap for crop management via a 5-pillar architecture. Since the system serves users who may have limited technical literacy and operate in rural areas with unstable networks, it is essential to perform thorough testing to eliminate UI friction, ensure accessible authentication, validate robust offline capabilities, and maintain data integrity.

- **TEST PLAN (LOGIN MODULE):**

Test Plan: (Android Application For Krushi Sakha)

Version: 1.0

Test Plan ID: TP_LOGIN_001

1. Introduction

This test plan verifies that the **registration and login module** of the Krushi Sakha application works correctly. It ensures that new users can create an account successfully and existing users can log in with valid credentials. Invalid inputs are handled properly with appropriate error messages. The plan also validates data storage, input validation, system reliability, and user-friendly navigation.

2. Test Items

- Registration form UI elements – input fields, Login button, error messages
- Login form UI elements – email, password, login button
- Name, Email, Password validation
- Form submission and Firebase/database storage
- Error message display for invalid or empty inputs
- Successful login and registration confirmation
- Navigation after successful authentication

3. Features to be Tested

- Valid registration with correct data
- Invalid registration with wrong inputs (email, password)
- Empty field validation
- Duplicate email/phone validation
- Password strength validation
- Successful login with valid credentials
- Invalid login with wrong credentials
- Navigation to main app after login

4. Features Not to be Tested

- Crop information modules
- Offline functionality
- Admin panel
- Reports and analytics

5. Approach

- Functional and non-functional testing
- Positive and negative test cases for login and registration
- Manual testing
- Backend verification using Firebase Console
- UI validation for readability and accessibility

6. Pass / Fail Criteria

- All high-priority test cases must pass
- Minimum 95% requirement coverage
- No critical defects remaining
- Expected results match actual results
- All major defects fixed and retested

7. Suspension Criteria and Resumption Requirements

- Testing stops if app crashes or critical bugs occur
- Testing resumes after bug fixing and stabilization

8. Test Deliverables

- Test plan document
- Test cases document
- Bug/defect report
- Final test summary report

9. Test Tasks

- Prepare test plan and test cases
- Execute test cases
- Identify and report defects
- Retest fixed defects
- Prepare final testing report

10. Test Environment

- Operating System: Android 10+
- Devices: Low-end and mid-range smartphones
- Frontend: Android Studio (Java/Kotlin)
- Backend: Firebase Authentication / Realtime Database
- Tools: Firebase Console
- Network: Internet connection required for registration/login

11. Responsibilities

- Tester: Execute tests and report defects
- Developer: Fix defects
- Project Guide: Review and approve
- User (Farmer): Provide usability feedback

12. Staffing and Training

- Knowledge of Android app testing and Firebase authentication required
- Testers should understand usability needs for rural farmers

13. Scheduling

- Project deadline: 17/04/26

14. Risks

- Poor internet connectivity affecting registration/login
- Limited user technical knowledge
- Incorrect input data by users
- Device compatibility issues

15. Approvals

- Project Guide
- Team Members

- **Test Cases For Login functionality:-**

Table 7.3.1 Test Cases For Login

Test Case ID	Objective	Steps (Input)	Expected Output	Actual Output	Status
Tc_01	Check Farmer login with valid credentials	Email="abc80@gmail.com", password=85673333	Login Successfully	Login Successfully	Pass
Tc_02	Check Login with empty fields	Email= " ", Password = " "	System Shows Message "Enter Credentials"	System Shows Message " Enter Credentials"	Pass
P	Login with wrong password	Email="abc80@gmail.com", Password=" Enter wrong password"	System Shows Message "Incorrect Password"	System Shows Message "IncorrectPassword"	Pass
Tc_04	Login with wrong Email	Email=" Enter Wrong Email " Password=" 85673333"	System Shows Message "Incorrect Email"	System Shows Message "Incorrect Email"	Pass
Tc_05	Check Login With Unregistered Email	1. Open Login. Enter Email not In Database. 3. Enter Password	User Not Found	Same as Expected	Pass
Tc_06	Check Invalid Email Format Validation	Email=" abc---_.com" , Password= "85673333"	System Shows Message "Enter Valid Email"	Login Successfully	Fail
Tc_07	Check Empty Email&PasswordValidation	Email = " ", Password = " "	Email & Password required	No Validation Message Show	Fail

Test Case ID	Objective	Steps (Input)	Expected Output	Actual Output	Status
Tc_08	Check Wrong Password	Email = “ amaypuranik789@gmail.com”, Password = “ Wrong Password”	Wrong Password Message Displayed	Login Successfully	Fail

- **Defect Report For Login Module:**

Table 7.3.2 Defect Report (Login Module)

Defect ID:	DR_LOGIN _0001
Project Name:	Krushi Sakha Mobile Application
Product:	Android Mobile App
Release Version:	v1.0
Module:	Login Page
Detected Build Version:	v1.1
Summary:	System allows login attempts with invalid email format
Description:	During testing of the login module, it was observed that the system does not validate email format properly. When the user enters an invalid email (e.g., "abc@") along with a password, the system accepts the input and proceeds with the login may lead to incorrect user data handling and potential security risks.
Expected Result:	System should display an error message: "Enter valid email" and block login attempt.
Actual Result:	System accepts invalid email and attempts login.
Severity:	Major
Priority:	High
Reported By:	Test Engineer
Status:	Open

7.4 Test Plan

This test plan document provides a detailed description of the testing strategy, objectives, scope, and resources required for testing the Krushi Sakha Android application. The main purpose of this plan is to ensure that the system meets all functional and non-functional requirements and operates reliably, efficiently, and smoothly for its target audience of agricultural workers.

Krushi Sakha is designed to provide farmers with a comprehensive, scientific roadmap for crop management via a 5-pillar architecture. Since the system serves users who may have limited technical literacy and operate in rural areas with unstable networks, it is essential to perform thorough testing to eliminate UI friction, ensure accessible authentication, validate robust offline capabilities, and maintain data integrity.

- **TEST PLAN (FUNCTIONALITY MODULE):**

Version: 1.0

Test Plan ID:TC_KS_001

1. Introduction

This test plan focuses on verifying the important functionality of the Krushi Sakha application, a digital companion for farmers. It ensures that core features such as crop cultivation guidance, offline access, stage-wise crop management, and usability work correctly. The plan aims to validate system reliability, accuracy of agricultural data, and accessibility for rural users.

2. Test Items

- Crop selection and stage-wise guidance (Pre-Sowing, Sowing, Nutrient & Water, Protection, Storage)
- Offline access for downloaded crop data
- Login and registration functionality
- Navigation between crop sections
- UI/UX readability (high-contrast theme for sunlight)
- Data storage and retrieval from Firebase .

3. Features to be Tested

- Accurate crop guidance for all included crops
- Stage-wise instructions display correctly
- Offline access works after data download
- Login/registration with valid and invalid inputs
- Smooth navigation between different crop modules
- Responsiveness of the app on different Android devices
- High-contrast UI visibility in sunlight
- Data saved in Firebase reflects correctly in the app.

4. Features Not to be Tested

- Admin panel
- Reports and analytics

5. Approach

- Functional testing of crop modules and navigation
- Positive and negative testing of login/registration
- Offline testing by disabling internet after data download
- UI/UX testing for readability and single-hand use
- Data validation with Firebase database
- Manual testing on multiple devices and screen sizes.

6. Pass / Fail Criteria

- All high-priority test cases must pass
- Minimum 95% requirement coverage
- No critical defects should remain
- Actual results must match expected results
- All major defects fixed and retested before release .

7. Suspension Criteria and Resumption Requirements

- Testing stops if the app crashes or key features fail
- Resumes after critical defects are fixed and the system stabilizes

8. Test Deliverables

- Test plan document
- Test cases document
- Defect/bug report
- Final test summary report

9. Test Tasks

- Prepare test plan and test cases for important functionality
- Execute tests and report defects
- Retest fixed issues
- Validate offline and online functionality
- Prepare final testing report .

10. Test Environment

- Operating System: Android 10+
- Devices: Low-end to mid-range smartphones
- Frontend: Android Studio (Java/Kotlin)
- Backend: Firebase Realtime Database / Firebase Authentication
- Network: Internet required for login and initial data download
- Tools: Firebase Console, manual testing on devices

11. Responsibilities

- QA Lead: Review test artifacts
- Test Engineer: Execute test cases, report defects
- Developer: Fix defects
- Project Guide: Approve release.

12. Staffing and Training

- Testers should understand crop guidance workflows, Android app testing, and Firebase
- Knowledge of usability requirements for rural farmers is recommended

13. Scheduling

- Project deadline: 17/04/26

- **14. Risks**
- Poor network affecting initial data download
- Limited user technical skills
- Device fragmentation (different screen sizes, resolutions)
- Data inconsistency in Firebase

15. Approvals

- Project Guide
- Team Members

- **Test Cases For System functionality:-**

Table 7.4.1 Test Cases For System Functionality

Test Case ID	Objective	Steps (Input)	Expected Result	Actual Result	Status
TC_KS_01	Validate Farmer/Admin Dashboard	After Login/Register Home page displayed	App opens without crash and home screen is displayed	App opened successfully and home screen displayed	Pass
TC_KS_02	Validate Crop Selection	Select any crop from list	Selected crop details should open	Wheat crop details opened correctly	Pass
TC_KS_03	Check Pre-Sowing Info	Click on Pre-Sowing section	Correct Pre-Sowing information displayed	Pre-Sowing information displayed properly	Pass
TC_KS_04	View Sowing Management	Click on Sowing section	Correct Sowing information displayed	Sowing management details displayed correctly	Pass
TC_KS_05	Validate Nutrient & Water Info	Open Nutrient & Water section	Proper schedule/details displayed	Nutrient and water schedule shown correctly	Pass
TC_KS_06	Check Crop Protection	Open Crop Protection section	Disease and pest control info displayed	Crop protection details displayed correctly	Pass

Test Case ID	Objective	Steps (Input)	Expected Result	Actual Result	Status
TC_KS_07	Check Storage Info	Open Storage section	Proper storage guidance displayed	Storage information displayed properly	Pass
TC_KS_08	Verify Offline Functionality	Turn off internet and access data	Previously loaded crop data should be accessible	Data not accessible without internet successfully	Fail
TC_KS_09	Validate Navigation	Move between different sections	Smooth navigation without lag or crash	Navigation smooth without any issue	Pass
TC_KS_10	Verify Data Storage	Check Firebase after registration	Data correctly stored in database	Data stored correctly in Firebase	Pass

- **Defect Report For System Functionality:**

Table 7.3.2 Defect Report for System Functionality

Defect ID:	DR_SYSTEMFUNC _0001
Project Name:	Krushi Sakha Mobile Application
Product:	Android Mobile App
Release Version:	v1.0
Module:	System Functionality
Detected Build Version:	v1.1
Summary:	Multiple issues found in registration, login, crop module, offline access, navigation, and UI visibility.
Description:	During testing of Krushi Sakha, the following defects were observed across 10 test cases: 1.Offline Mode: i) Verify Offline Functionality
Expected Result:	Previously loaded crop data should be accessible
Actual Result:	Data not accessible without internet successfully
Severity:	High
Priority:	High
Status:	Fixed

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

Krushi Sakha is a practical Android application designed for the agricultural community, specifically targeting farmers who need scientific crop cultivation guidance without the barriers of internet connectivity or user registration. It helps farmers access complete, stage- wise crop information through its five-pillar structure from land preparation before sowing all the way to proper post-harvest storage. With strong offline functionality, simple real-time search, category and seasonal filtering, a large font accessibility mode, and a high-contrast interface designed for outdoor use, the system makes scientific agriculture accessible to every farmer with a basic Android smartphone.

8.2 FUTURE SCOPE

The future scope of Krushi Sakha includes adding voice-based crop search using the Android SpeechRecognizer API, multilingual support for Marathi and Hindi, real-time weather API integration for crop-specific advisories, AI-based pest identification using TensorFlow Lite, and expansion of the crop database to cover 100+ crops including medicinal plants and horticulture. An iOS version using Flutter or React Native is also planned.

CHAPTER 9

REFERENCE AND BIBLIOGRAPHY

1. REFERENCE

- **Books**

1. Software Engineering: A Practitioner's Approach, Eighth Edition — Roger S. Pressman, Ph.D. and Bruce R. Maxim, Ph.D.
2. Android Programming: The Big Nerd Ranch Guide, Fourth Edition — Bill Phillips, Chris Stewart, Kristin Marsicano.
3. Head First Android Development, Third Edition — Dawn and David Griffiths. O'Reilly Media.
4. Firebase Essentials — Android Edition — Neil Smyth. Payload Media.

- **Research Papers**

1. Kumar, A., et al. "Mobile Application for Crop Advisory using Firebase and Android." International Journal of Research in Engineering, Science and Management, 2022.
2. Patil, R. V. "Design of Offline-capable Agricultural Advisory System for Rural India." IRJET, Vol. 9, Issue 4, April 2022.
3. Sharma, D., Verma, P. "AgriTech Mobile Applications: A Comparative Study of Existing Farming Advisory Systems in India." IEEE CONIT 2023.

- **Websites**

1. Android Developer Documentation — https://developer.android.com/docs
2. Firebase Documentation (Realtime Database, Storage, Authentication) — https://firebase.google.com/docs
3. Glide Image Loading Library — https://bumptech.github.io/glide/
4. Java Android Tutorials — https://www.tutorialspoint.com/android/index.htm
5. XML Layout Reference — https://developer.android.com/reference/android/view/View
6. ICAR Crop Production Guidelines — https://icar.org.in/
7. AgMarkNet Market Prices — https://agmarknet.gov.in/
8. National Horticulture Board Crop Profiles — https://nhb.gov.in/
9. Maharashtra Agriculture Department Crop Data https://krishi.maharashtra.gov.in/

2. BIBLIOGRAPHY

Existing Krushi Sakha review:

DEPARTMENT OF COMPUTER ENGINEERING

S.S.V.P.S's Bapusaheb Shivajirao Deore Polytechnic, Dhule.

(Institute Code: 0059)

Academic Year: 2025-2026